

Fire and Smoke Detection

Model selection report (Stage 1)

Three backbones and two detection heads compared over 40 training epochs

07 October 2026

Summary. The task is an industrial fire and smoke detector for fixed cameras that must run around the clock with very few false alarms. Before investing in one design, four models were trained under identical conditions for 40 epochs: three with the same Faster R-CNN detector on different backbones (DINOv3 ViT-S, MobileNetV3-Large, ResNet-18), and one with the FCOS detector. **DINOv3 + Faster R-CNN was the most accurate on every measure** (validation mAP@0.5 0.7312, test 0.7196) and kept false alarms lowest at the same detection rate, so the project continues with it. MobileNetV3 is kept as the candidate for a faster version.

Model	Best val mAP@0.5	Test mAP@0.5	Parameters	CPU time / frame
DINOv3 ViT-S + Faster R-CNN	0.7312 (epoch 38)	0.7196	39.3M	894-912 ms
MobileNetV3-L + Faster R-CNN	0.7056 (epoch 39)	0.6968	20.0M	346-385 ms
ResNet-18 + Faster R-CNN	0.6567 (epoch 39)	0.6477	27.9M	414 ms
DINOv3 ViT-Ti + FCOS	0.6212 (epoch 39)	0.6125	6.95M	338-398 ms

All four trained for 40 epochs on the same data with the backbone frozen. CPU time measured on the project laptop (batch 1, 640 px).

Notes on measurement are in Appendix B.

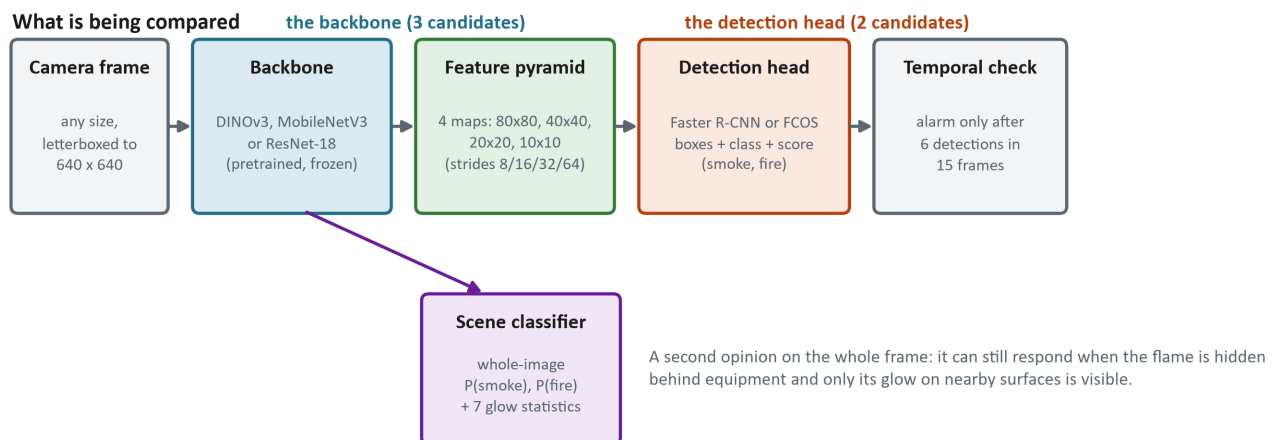
1. Problem statement

An industrial site needs fire and smoke detected automatically from its existing surveillance cameras. The conditions make this harder than ordinary fire detection in photographs:

#	Problem to solve	What it demands from the system
1	Cameras are mounted high, in poor industrial lighting; fire is a bright light source	Robust features under low light and glare; training with low-light and infrared-style images
2	The system runs 24 hours a day, 7 days a week	False alarms must be very rare. The false-alarm rate is measured alongside accuracy, and an alarm needs repeated detections over time
3	A fire can be hidden behind equipment	Detect it from the light it casts on nearby surfaces: a whole-image classifier plus glow statistics
4	Smoke must be detected when it is visible (invisible smoke is out of scope)	Two classes, smoke and fire, each with its own boxes
5	Fires can be small or far from the camera	Features at several scales (a feature pyramid)
6	Many cameras (about 70), at least 10 frames per second overall	Speed and model size matter as well as accuracy; a lighter variant must be possible
7	Work first on images, then on video	Train and benchmark on a labelled image dataset; add temporal confirmation for video

These requirements lead to one central design question, which this report answers with measurements: **which backbone and which detection head should the detector be built on?**

2. Approach



The detector. Stage 1 compares three choices for the backbone and two for the detection head; everything else stays the same.

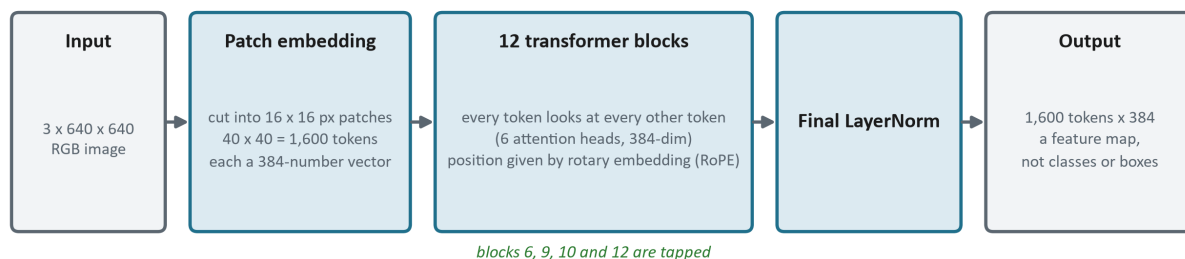
- **Backbone:** a pretrained network that turns the image into features. Candidates: DINOv3 ViT-S/16, MobileNetV3-Large, ResNet-18.
- **Feature pyramid:** four feature maps at different scales, so both large and small fires are covered.
- **Detection head:** turns features into boxes with a class and a score. Candidates: Faster R-CNN (two-stage) and FCOS (one-stage).
- **Scene classifier:** a second, whole-image output, P(smoke) and P(fire), that also reads seven 'glow' statistics. It addresses problem 3 (hidden fire).
- **Stage 1 rule:** the backbone is *frozen*. Only the pyramid, the head and the scene classifier are trained. This tests how good each backbone's ready-made features are for fire and smoke.

3. The backbone candidates

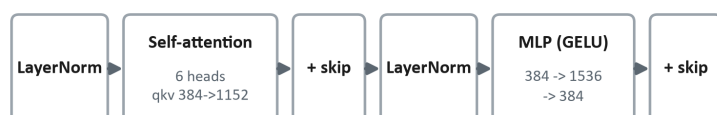
3.1 DINOv3 ViT-S/16

DINOv3 is a Vision Transformer trained by Meta AI with *self-supervised* learning: it learned from 1.689 billion unlabelled images by matching its own predictions across different views of the same image (self-distillation with no labels, hence DI-NO). Because no labels were used, its features are general rather than tuned to one classification task. ViT-S/16 is the small member of the family.

DINOv3 ViT-S/16 - 21.6M parameters, 12 transformer blocks



Inside one transformer block (repeated 12 times)



What our detector takes from it

The 4 tapped maps (all 40 x 40) are resampled into a pyramid: p3 80 x 80, p4 40 x 40, p5 20 x 20, p6 10 x 10; 256 channels each (as in ViTDet). Neck: 3.02M trainable parameters.

Pretraining: self-supervised (no labels) on LVD-1689M, 1.689 billion images. One global receptive field from the first block.

Input	RGB image, 3 x 640 x 640, normalised
How it works	The image is cut into 16 x 16-pixel patches (40 x 40 = 1,600 patches). Each patch becomes a 384-number vector, a 'token'. Twelve transformer blocks then let every token exchange information with every other token (self-attention), so each one comes to describe its patch in the context of the whole image.
Output	1,600 tokens x 384 values: a feature map at 1/16 resolution. It gives no classes or boxes by itself; a detection head is added on top.
Layers	12 transformer blocks. Each has 2 LayerNorms, a 6-head self-attention layer and a 2-layer MLP (384 -> 1,536 -> 384, GELU), with skip connections. 48 linear layers in total, plus one convolution for the patch embedding.
Parameters	21.6M in the backbone (frozen in stage 1). Whole detector: 39.3M, of which 17.7M are trained.
How we train it	Backbone frozen. Four of its blocks (6, 9, 10, 12) are tapped and resampled into a 4-level pyramid (3.02M parameters), which feeds the detection head.

Advantages

- Strongest ready-made features: best accuracy with the backbone frozen
- Every token sees the whole image from the first block (global context), useful for smoke, which has no fixed shape
- Trained on 1,300 times more images than the two CNNs, with no labels
- Lowest false-alarm rate at the same detection rate in our tests

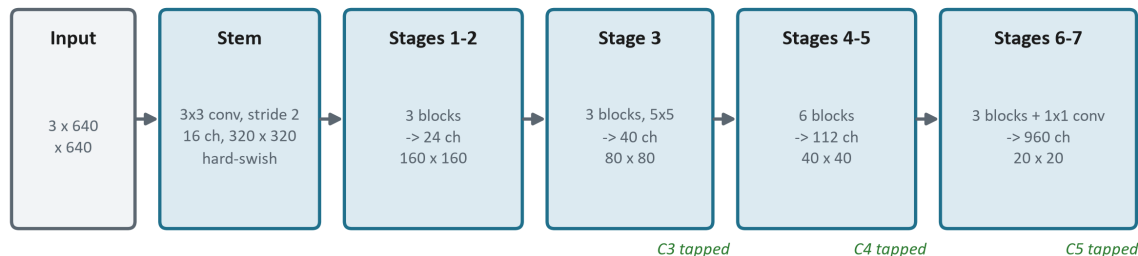
Disadvantages

- Slowest: about 900 ms per frame on CPU, 2.5 times the CNNs
- Self-attention cost grows quickly with image size
- Produces one scale only, so a pyramid has to be built from it
- Largest model (39.3M parameters with the detector)

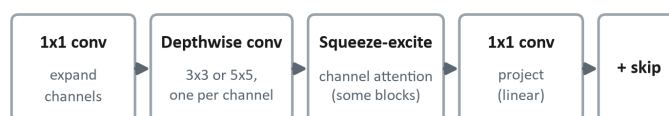
3.2 MobileNetV3-Large

MobileNetV3 (Google, 2019) is a convolutional network designed for phones and embedded devices. Its structure was found partly by automated architecture search, with the goal of the best accuracy for a given amount of computation. It was pretrained with labels on ImageNet-1k (1.28 million images, 1,000 classes).

MobileNetV3-Large - 2.9M parameters, 15 inverted-residual blocks, 62 convolutions



Inside one inverted-residual block



What our detector takes from it

C3, C4, C5 (40 / 112 / 960 channels) go through the same Feature Pyramid Network as ResNet-18: p3 80 x 80, p4 40 x 40, p5 20 x 20, p6 10 x 10; 256 channels each. Neck: 2.06M trainable parameters.

Pretraining: supervised classification on ImageNet-1k. Designed by architecture search for phones: depthwise convolutions make it cheap.

Input	RGB image, 3 x 640 x 640, normalised
How it works	A stack of 'inverted residual' blocks. Each block widens the channels with a 1 x 1 convolution, filters each channel separately with a cheap <i>depthwise</i> convolution, optionally re-weights channels (squeeze-and-excite), then narrows again. The image gets smaller and the channels more numerous stage by stage.
Output	Feature maps at several scales. We use three: 80 x 80 (40 channels), 40 x 40 (112 channels) and 20 x 20 (960 channels).
Layers	15 inverted-residual blocks in 7 stages, 62 convolutions; hard-swish and ReLU activations.
Parameters	2.95M in the backbone (frozen in stage 1). Whole detector: 20.0M, of which 17.1M are trained.
How we train it	Backbone and its BatchNorm statistics frozen. The three maps go through a standard Feature Pyramid Network (2.06M parameters) that also adds a fourth, coarser level.

Advantages

- Smallest backbone (2.9M parameters) and about 2.5 times faster than DINOv3 on CPU
- Gives several scales naturally, which suits a feature pyramid
- Second-best accuracy, only 2.3-2.6 points of mAP behind DINOv3
- Well supported on edge hardware

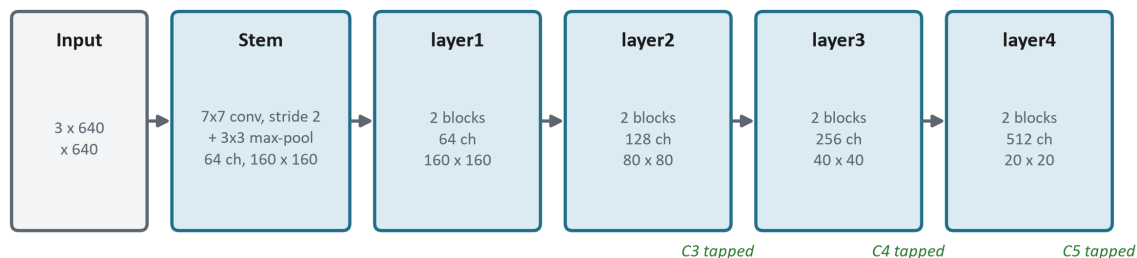
Disadvantages

- Features come from a classification task, so they are less general
- Each layer sees only a local neighbourhood
- More false alarms than DINOv3 at the same detection rate (0.87% against 0.37% on validation)
- Lower smoke accuracy than DINOv3

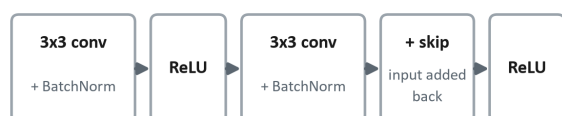
3.3 ResNet-18

ResNet (Microsoft Research, 2015) introduced the *skip connection*: each block adds its input back to its output, which made deep networks trainable. ResNet-18 is the smallest standard member and a common baseline. It was pretrained with labels on ImageNet-1k.

ResNet-18 - 11.2M parameters, 8 residual blocks, 20 convolutions



Inside one residual block (BasicBlock)



What our detector takes from it

C3, C4, C5 (128 / 256 / 512 channels) go through a standard Feature Pyramid Network: p3 80 x 80, p4 40 x 40, p5 20 x 20, and p6 10 x 10 pooled from p5; 256 channels each. Neck: 2.00M trainable parameters.

Pretraining: supervised classification on ImageNet-1k (1.28 million labelled images, 1,000 classes). BatchNorm statistics kept frozen.

Input	RGB image, 3 x 640 x 640, normalised
How it works	A 7 x 7 convolution and a pooling layer shrink the image to a quarter. Four stages follow, each with two residual blocks of two 3 x 3 convolutions. Each stage after the first halves the resolution and doubles the channels (64, 128, 256, 512).
Output	We use three maps: 80 x 80 (128 channels), 40 x 40 (256 channels) and 20 x 20 (512 channels).
Layers	8 residual blocks, 20 convolutions (17 main + 3 on shortcuts), each followed by BatchNorm; ReLU activations. The name counts 17 convolutions + 1 classifier layer, which we remove.
Parameters	11.2M in the backbone (frozen in stage 1). Whole detector: 27.9M, of which 16.8M are trained.
How we train it	Same as MobileNetV3: backbone and BatchNorm statistics frozen, a standard Feature Pyramid Network on top (2.00M parameters).

Advantages

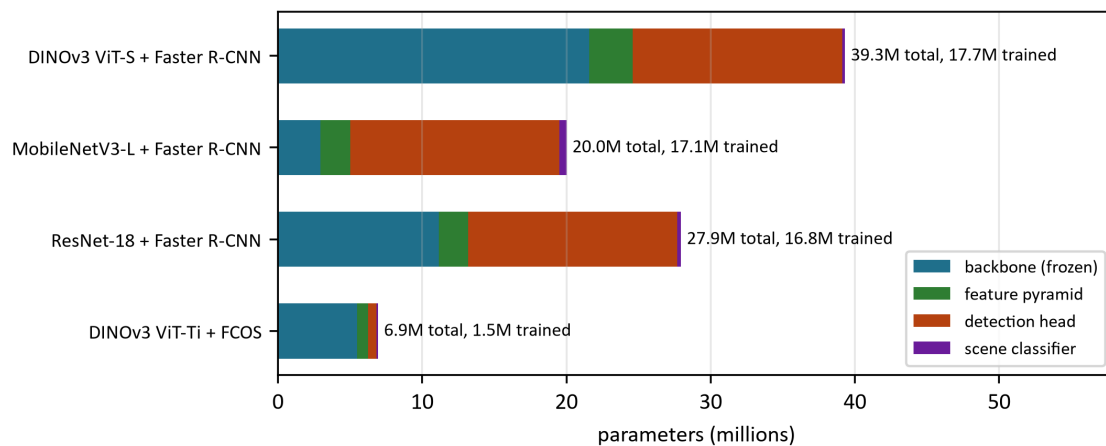
- Simple and very well understood; supported everywhere
- Regular 3 x 3 convolutions run efficiently on most hardware
- Several scales naturally

Disadvantages

- Lowest accuracy of the three backbones (validation mAP 0.6567)
- Slower than MobileNetV3 here (414 ms against about 365 ms)
- Misses many more fires at a low false-alarm rate: 68% detected against 87% for the other two
- Its frozen features fit the task worst: training loss stays highest

3.4 Backbones side by side

	DINOv3 ViT-S/16	MobileNetV3-Large	ResNet-18
Type	Vision Transformer	Convolutional (mobile)	Convolutional (residual)
Building block	12 transformer blocks	15 inverted-residual blocks	8 residual blocks
Backbone parameters	21.6M	2.95M	11.2M
Pretraining	self-supervised, 1.689 billion images	supervised, ImageNet-1k	supervised, ImageNet-1k
What each layer sees	the whole image	a local neighbourhood	a local neighbourhood
Feature scales produced	one (pyramid built from it)	several	several
Detector total / trained	39.3M / 17.7M	20.0M / 17.1M	27.9M / 16.8M

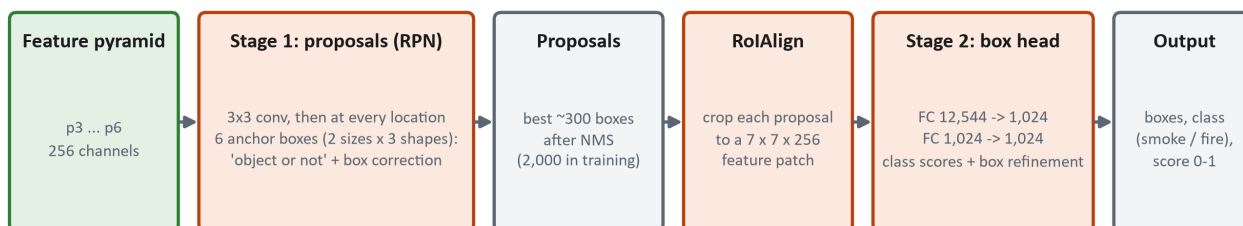


Where the parameters are. In all three Faster R-CNN detectors most trained parameters sit in the detection head, so the three train nearly the same amount; what differs is the frozen backbone.

4. The detection-head candidates

4.1 Faster R-CNN (two-stage)

Faster R-CNN - two-stage detector: first propose regions, then examine each one



Parameters (ours): RPN 0.60M, box head 13.91M -> 14.5M, all trained.

Training losses (4): RPN objectness (binary cross-entropy), RPN box (smooth L1), box-head class (cross-entropy over background / smoke / fire), box-head box (smooth L1).

Anchors: sizes 16-32 / 48-96 / 128-192 / 256-384 px on p3 / p4 / p5 / p6, shapes 1:2, 1:1, 2:1.
The number of proposals depends on the image, so the middle of the network has no fixed shape.

Input	The 4-level feature pyramid (256 channels per level)
How it works	Stage 1 , the Region Proposal Network, slides over every location of every level and, for 6 preset 'anchor' boxes there, predicts whether an object is present and how to adjust the box. The best ~300 are kept as proposals. Stage 2 crops the features of each proposal to a fixed 7 x 7 patch (RoIAlign) and passes them through two fully-connected layers, which decide the class and refine the box.
Output	Boxes, each with a class (smoke or fire) and a confidence score from 0 to 1
Parameters	14.5M, all trained: proposal network 0.60M, box head 13.91M
Training losses	Four: objectness and box loss for the proposal network, class and box loss for the box head

Advantages

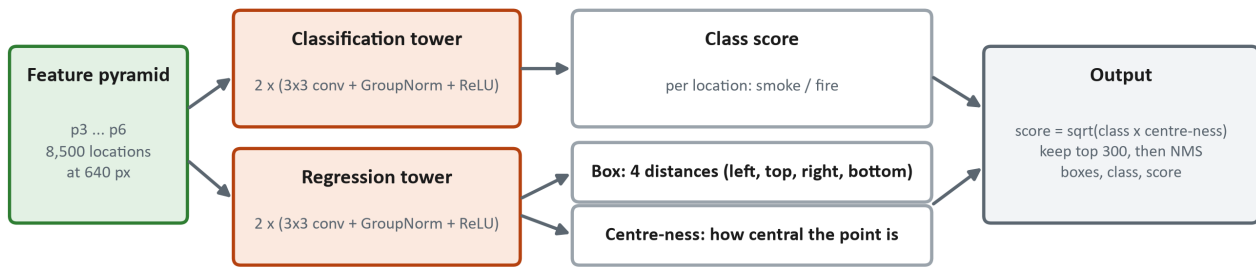
- Most accurate in our tests
- Every detection is examined twice, so confident detections are well separated from doubtful ones: a threshold of 0.90 is usable
- Mature and widely used; handles objects of very different sizes

Disadvantages

- Large head (14.5M parameters)
- The number of proposals changes from image to image, so the network cannot be exported as one fixed-shape graph for batching many cameras (tested: export fails for batch sizes above 1)
- Needs anchor sizes to be chosen

4.2 FCOS (one-stage, anchor-free)

FCOS - one-stage, anchor-free detector: every location predicts directly



Parameters (ours): 0.60M, all trained.

Training losses (3): class (focal loss, handles the many empty locations), box (GloU loss), centre-ness (binary cross-entropy).

No anchors and no proposals: the network always outputs the same fixed-size tensors, so frames from many cameras can be processed together as one batch.

Input	The same 4-level feature pyramid
How it works	No proposals and no anchors. Every location of every level (8,500 locations at 640 px) directly predicts three things through two small convolution towers: a class score, the distances to the four sides of the box, and a 'centre-ness' value that lowers the score of points near an object's edge.
Output	Boxes, class and score. The score is $\text{sqrt}(\text{class score} \times \text{centre-ness})$.
Parameters	0.60M, all trained (2 convolutions per tower, 128 channels in our configuration)
Training losses	Three: focal loss (class), GloU loss (box), binary cross-entropy (centre-ness)

Advantages

- Simple: no anchors, no proposals, 24 times fewer head parameters than Faster R-CNN
- Always outputs the same fixed-size tensors, so many cameras can be batched (export verified)
- Fewer settings to tune

Disadvantages

- Lower accuracy in our trained configuration
- Scores are compressed (they never exceed about 0.75), so its alarm threshold has to sit at 0.55
- Not faster by itself: on the same backbone it took 924 ms against 884 ms for Faster R-CNN

5. How the comparison was run

The comparison is only meaningful if nothing changes except the part being compared. All four models used the same data, the same splits, the same augmentation and the same training settings.

Data

D-Fire dataset split	Images	With no fire or smoke	Smoke boxes	Fire boxes
Training	13,776	6,223	7,660	9,659
Validation (results after every epoch)	3,445	1,610	1,890	2,155
Test (used once, at the end)	4,306	2,005	2,311	2,878

About 45% of the images contain no fire or smoke. They are what makes it possible to measure false alarms.

Training settings (identical for all four models)

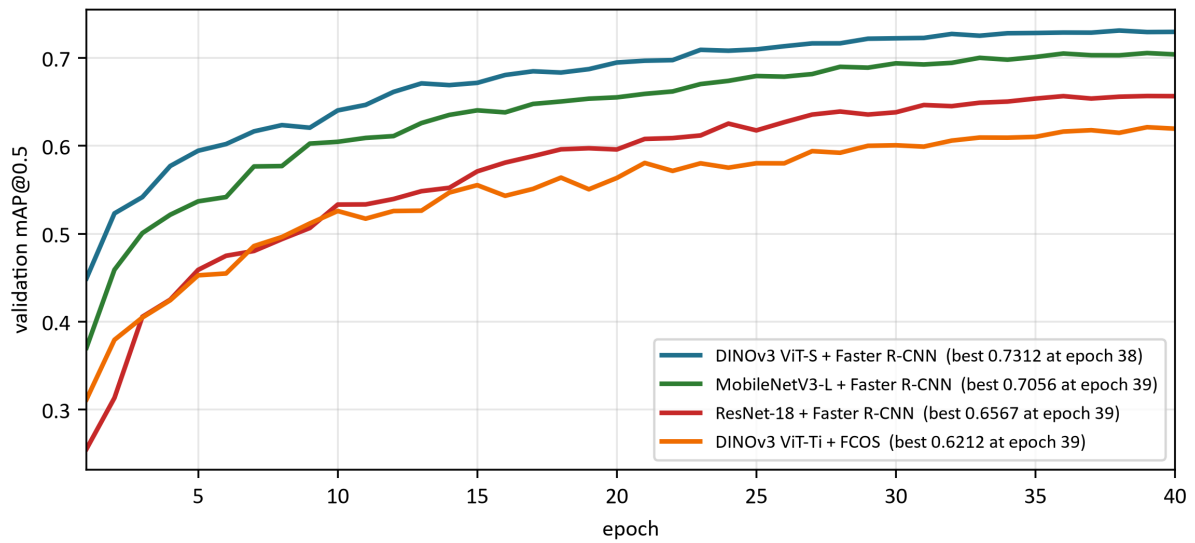
Epochs	40 (the FCOS model ran 45; its first 40 are reported here)
Backbone	Frozen. Only the feature pyramid, the detection head and the scene classifier are trained
Optimiser	AdamW, learning rate 0.0001, weight decay 0.0001
Schedule	500-step warm-up, then cosine decay to 1% of the starting rate
Batch and precision	Batch size 8, mixed precision (16-bit), gradient clipping at 10
Input size	640 x 640 pixels (aspect ratio kept, padded)
Augmentation	Low-light simulation, infrared-style grey images, hiding part of a flame, random crop, colour jitter, flip, motion blur
Model selection	The epoch with the best validation mAP@0.5 is kept
Hardware	Kaggle T4 GPU, about 8-11 minutes per epoch

What is measured after every epoch

- **AP@0.5** for smoke and for fire: how accurately boxes are found, counting a box as correct when it overlaps the labelled box by at least 50%. **mAP@0.5** is their average and is the main benchmark.
- **mAP@0.5:0.95**: the same averaged over stricter overlap requirements.
- **Detected at <= 1% false alarms**: the confidence threshold is raised until at most 1% of the fire-free images trigger a detection, then the share of fire/smoke images still detected is reported. This is the number that matters for an alarm system.
- **Training loss**: how well the model fits the training images.

6. Results: backbone comparison

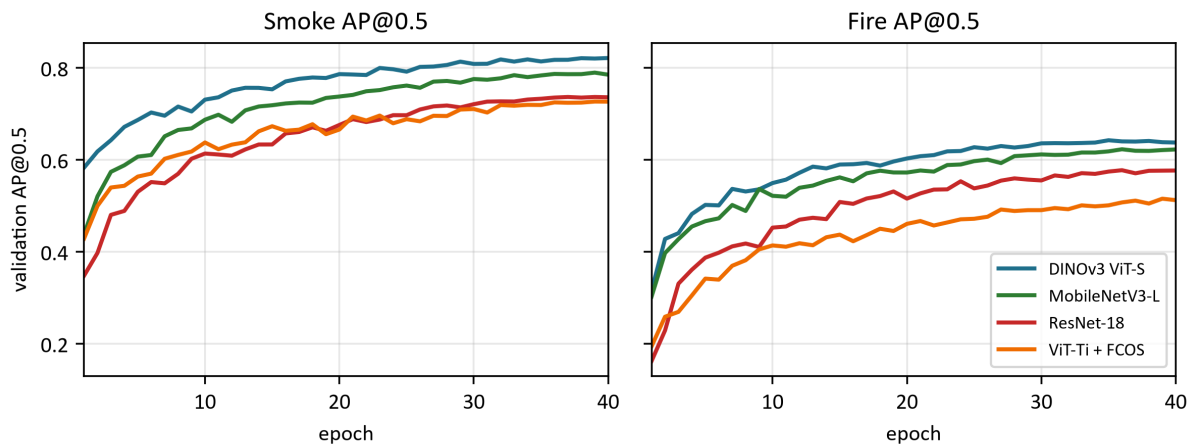
Three models, identical except for the backbone, each with the Faster R-CNN head.



Validation mAP@0.5 after each of the 40 epochs. The order DINOv3 > MobileNetV3 > ResNet-18 holds at every epoch. The FCOS model is shown for reference and discussed in section 7.

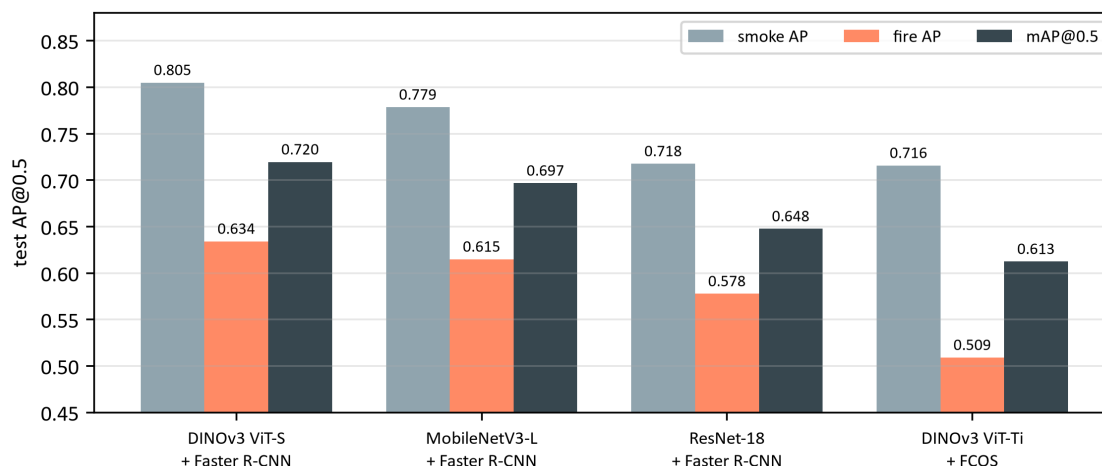
Validation mAP@0.5 at epoch	1	5	10	20	30	40	Best
DINOv3 ViT-S + Faster R-CNN	0.4494	0.5945	0.6403	0.6948	0.7224	0.7297	0.7312 (ep 38)
MobileNetV3-L + Faster R-CNN	0.3705	0.5369	0.6046	0.6552	0.6939	0.7040	0.7056 (ep 39)
ResNet-18 + Faster R-CNN	0.2553	0.4590	0.5333	0.5959	0.6382	0.6566	0.6567 (ep 39)
DINOv3 ViT-Ti + FCOS	0.3117	0.4528	0.5259	0.5635	0.6007	0.6196	0.6212 (ep 39)

Results for every epoch of every model are in Appendix A.



Per class. Fire is harder than smoke for every model; DINOv3 leads on both.

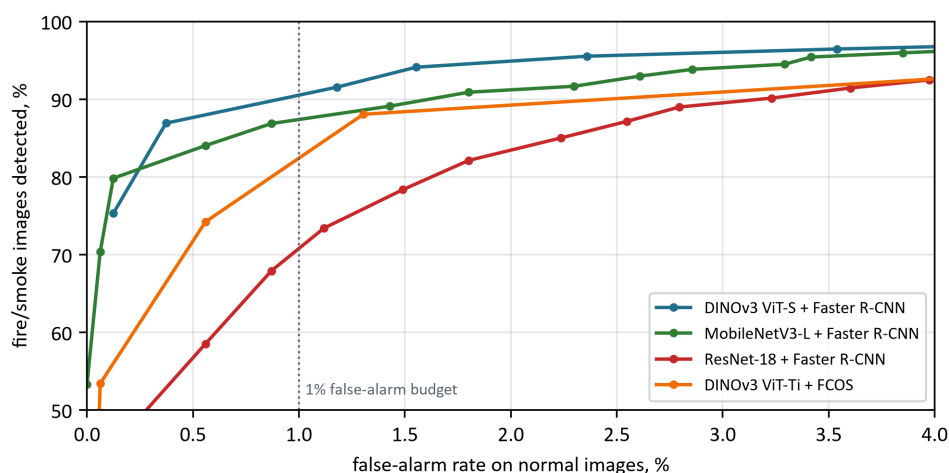
Test set



Test split (4,306 images)	Smoke AP	Fire AP	mAP@0.5	mAP@0.5:0.95
DINOv3 ViT-S + Faster R-CNN	0.8050	0.6342	0.7196	0.3627
MobileNetV3-L + Faster R-CNN	0.7786	0.6150	0.6968	0.3334
ResNet-18 + Faster R-CNN	0.7177	0.5778	0.6477	0.3046
DINOv3 ViT-Ti + FCOS	0.7158	0.5092	0.6125	0.2854

The test set was not used for training or for choosing the epoch. The ranking is the same as on validation.

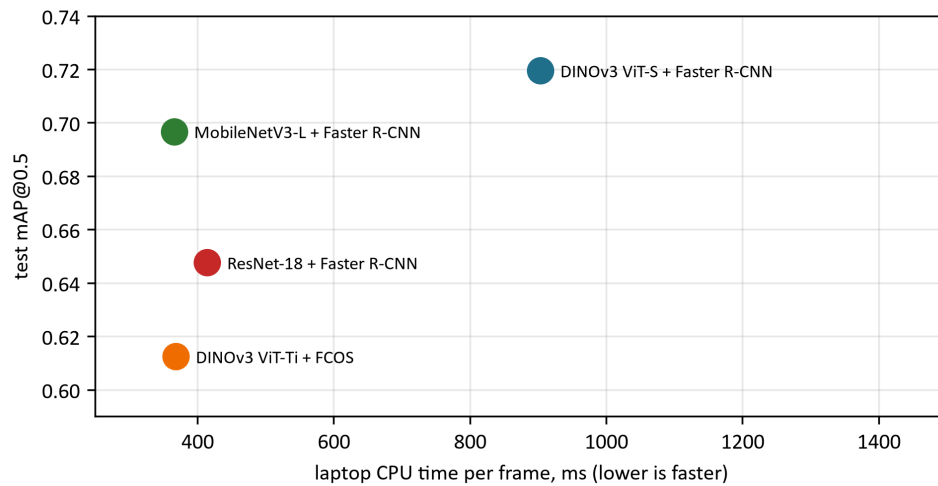
False alarms against detections



At <= 1% false alarms (validation)	Threshold	False-alarm rate	Fire/smoke images detected
DINOv3 ViT-S + Faster R-CNN	0.90	0.37%	86.9%
MobileNetV3-L + Faster R-CNN	0.88	0.87%	86.9%
ResNet-18 + Faster R-CNN	0.93	0.87%	67.9%
DINOv3 ViT-Ti + FCOS	0.55	0.56%	74.2%

DINOv3 and MobileNetV3 detect the same share of fire/smoke images, but DINOv3 does it with less than half the false alarms. ResNet-18 detects far fewer.

Speed



Accuracy against speed on the project laptop's CPU. MobileNetV3 is the best trade-off for a fast version; ResNet-18 is both slower and less accurate than MobileNetV3.

- **DINOv3** is the most accurate (test mAP 0.7196) and the slowest (894-912 ms).
- **MobileNetV3** is 2.3 points behind on test and about 2.5 times faster.
- **ResNet-18** is 7.2 points behind and not faster than MobileNetV3.
- The three Faster R-CNN detectors train almost the same number of parameters (16.8M-17.7M), because most of them are in the head. The difference in results therefore comes from the frozen backbone's features.

7. Results: detection-head comparison

Two detectors were trained to completion: DINOv3 ViT-S with Faster R-CNN, and a lighter DINOv3 ViT-Ti with FCOS. FCOS was built as the deployable, light option, so this pair differs in **two** things at once: the head and the size of the backbone. The accuracy gap below therefore cannot be attributed to the head alone; the speed and export measurements, made on the same backbone, can.

	DINOv3 ViT-S + Faster R-CNN	DINOv3 ViT-Ti + FCOS
Detector type	two-stage, anchor-based	one-stage, anchor-free
Backbone / head parameters	21.6M / 14.5M	5.49M / 0.60M
Trained parameters	17.7M	1.46M
Best validation mAP@0.5 (40 epochs)	0.7312	0.6212
Test mAP@0.5 (smoke / fire)	0.7196 (0.805 / 0.634)	0.6125 (0.716 / 0.509)
Detected at <= 1% false alarms (validation)	86.9%	74.2%
Usable alarm threshold	0.90	0.55 (scores top out near 0.75)
CPU time per frame	894-912 ms	338-398 ms
Export as one batched graph (many cameras)	fails above batch size 1	works, verified

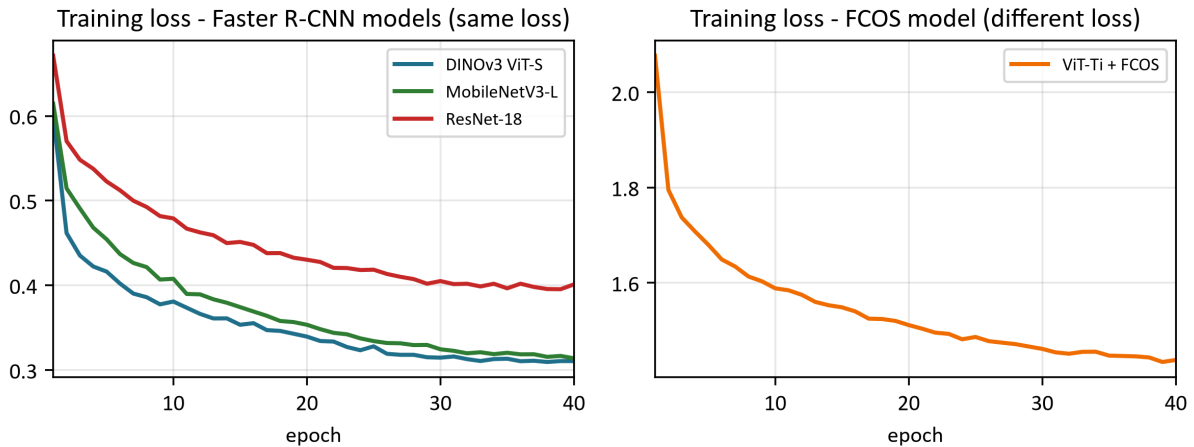
Does the head make the model faster? Measured on the same backbone

Configuration	CPU ms / frame	Parameters
ViT-S + Faster R-CNN	884	39.3M
ViT-S + FCOS, 4 convs	1177	-
ViT-S + FCOS, 2 convs	924	27.2M
ViT-S + FCOS, 2 convs, 128-ch pyramid	746	23.2M
ViT-Ti + FCOS, 2 convs, 128-ch pyramid	329	6.9M

Same DINOv3 ViT-S backbone in the first four rows. Swapping the head alone does not speed the model up; the backbone is about two thirds of the computation.

- **Accuracy.** Faster R-CNN's model is 10.7 points of test mAP ahead, and detects 13 points more fire/smoke images at the 1% false-alarm budget. Part of this comes from the smaller backbone.
- **Scores.** Faster R-CNN gives clearly separated scores, so a strict threshold (0.90) still keeps most detections. FCOS multiplies two numbers below 1, so its scores are compressed and the threshold must be set low and precisely.
- **Speed.** FCOS is not faster on the same backbone (924 ms against 884 ms). The light model is faster because of its smaller backbone.
- **Deployment.** FCOS's real advantage is the fixed-shape output, which allows frames from many cameras to be processed as one batch. Faster R-CNN cannot be exported that way.

8. Training behaviour: did any model overfit?



Training loss over 40 epochs. Faster R-CNN and FCOS use different loss functions, so their values are not comparable with each other, only their shapes.

Model	Train loss, epoch 1 -> 40	Val mAP, epoch 30 -> 40	Best epoch	Trained parameters
DINOv3 ViT-S + Faster R-CNN	0.601 -> 0.311	0.7224 -> 0.7297 (+0.7 pts)	38	17.7M
MobileNetV3-L + Faster R-CNN	0.614 -> 0.314	0.6939 -> 0.7040 (+1.0 pts)	39	17.1M
ResNet-18 + Faster R-CNN	0.671 -> 0.401	0.6382 -> 0.6566 (+1.8 pts)	39	16.8M
DINOv3 ViT-Ti + FCOS	2.077 -> 1.438	0.6007 -> 0.6196 (+1.9 pts)	39	1.46M

No model overfitted in 40 epochs. Overfitting would show as training loss still falling while validation accuracy turns downward. That did not happen for any of the four: validation mAP was still rising in the last ten epochs for every model, and the best epoch was 38 or 39 in every case.

Why there was no overfitting

- **The backbone is frozen.** The part of the network that could memorise the training images most easily is not being updated at all.
- **Strong augmentation.** Each image is seen in a different crop, brightness and colour every epoch.
- **45% of the training images contain no fire or smoke**, which constantly penalises false detections.
- **The best validation epoch is kept**, so a late decline would not have been used in any case.

What the smaller and weaker models show instead: underfitting

- **ResNet-18** ends with the highest training loss of the Faster R-CNN models (0.401 against 0.311 for DINOv3) while training almost the same number of parameters. With the same head and the same data, the only difference is the frozen features: they describe fire and smoke less well, so the head cannot fit even the training set as closely.
- **The FCOS model** trains only 1.46M parameters, twelve times fewer than the others, on a backbone half as wide. It was still improving fastest at the end (+1.9 points in the last ten epochs): it has too little capacity rather than too much.
- **Being small and fast did not cause overfitting here; it limited how much the model could learn.** The risk of overfitting appears later, when backbone blocks are unfrozen, and that is why the next stage uses a much lower learning rate for the backbone.

9. Decision: continue with DINOv3 + Faster R-CNN

Criterion	DINOv3 ViT-S	MobileNetV3-L	ResNet-18	DINOv3 ViT-Ti
Detection head	Faster R-CNN	Faster R-CNN	Faster R-CNN	FCOS
Validation mAP@0.5	0.7312	0.7056	0.6567	0.6212
Test mAP@0.5	0.7196	0.6968	0.6477	0.6125
Test fire AP	0.6342	0.6150	0.5778	0.5092
Detected at <= 1% false alarms	86.9%	86.9%	67.9%	74.2%
False-alarm rate at that point	0.37%	0.87%	0.87%	0.56%
CPU ms per frame	894-912	346-385	414	338-398
Batched export	no	no	no	yes

Reasons

- **1. It is the most accurate.** Best mAP on validation and on test, best on both smoke and fire, and its validation mAP was the highest of the three backbones at every one of the 40 epochs.
- **2. It gives the fewest false alarms for the same detections.** The central requirement is very few false alarms in 24/7 use. At the 1% budget DINOv3 detects 86.9% of fire/smoke images with a 0.37% false-alarm rate; MobileNetV3 needs 0.87% for the same detections.
- **3. Its features are good without being trained.** With all backbones frozen, DINOv3's features gave the best result. Self-supervised training on 1.689 billion images transfers better to fire and smoke than classification training on ImageNet.
- **4. Faster R-CNN gives clean scores.** A strict threshold (0.90) remains usable, which is what a low-false-alarm system needs. FCOS's compressed scores make its threshold fragile.
- **5. It has room to improve.** Its curve was still rising at epoch 40, and none of its backbone has been fine-tuned yet.

What this choice costs

- **Speed.** About 900 ms per frame on a laptop CPU, 2.5 times slower than MobileNetV3. Acceptable on a GPU, but it has to be addressed for 70 cameras.
- **No batched export** with Faster R-CNN.

Both costs concern deployment rather than detection quality, so accuracy is settled first. **MobileNetV3** is kept as the candidate for a fast version (close in accuracy, 2.5 times faster), and **FCOS** as the head that makes multi-camera batching possible. ResNet-18 is dropped: MobileNetV3 is both faster and more accurate.

10. Next goals

#	Goal	Purpose
1	Stage 2: fine-tune part of the backbone. Unfreeze the last 2 transformer blocks of DINOv3 with a learning rate 20 times lower than the head's	Adapt the features to fire and smoke without overfitting; raise accuracy further
2	Small and distant fires. Higher input resolution and scale augmentation (mosaic)	Fires far from the camera cover very few pixels
3	Hard-negative training. Add fire-coloured objects labelled as 'not fire': hard hats, hi-vis clothing, lamps, bright haze	Bring false alarms down further for 24/7 use
4	Video. Run on industrial footage with temporal confirmation (6 detections in 15 frames)	Measure false alarms per hour and time to alarm
5	A fast version for many cameras. MobileNetV3 backbone and/or FCOS head, exported for GPU batching	Meet the 70-camera, 10 fps requirement
6	Data from the site's own cameras, labelled with one consistent rule	Match the model to the real scene and lighting

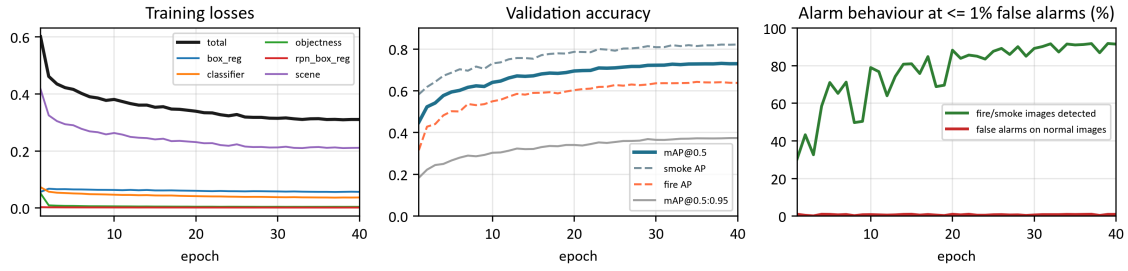
11. Project documents

Document	Content
PROJECT_LOG.md	Running log of every decision, measurement and change
Model selection report (this document)	Stage 1: problem, models, 40-epoch comparison, decision
Model selection slides	Ten-slide summary of this report
Training report	Dataset, architecture and training details of the DINOv3 detector
Technical and research report	Layer-by-layer architecture, loss functions and background research
Model comparison report	Faster R-CNN model against the light FCOS model in detail
Source code and training scripts	Training, evaluation, video inference and report builders

Appendix A. Results of every epoch

A.1 DINOv3 ViT-S + Faster R-CNN

DINOv3 ViT-S + Faster R-CNN - 40 epochs

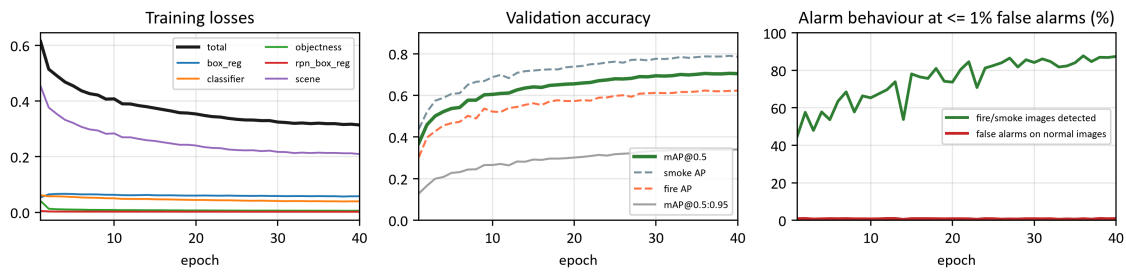


Epoch	Train loss	Smoke AP	Fire AP	mAP@0.5	mAP@.5:.95	Detected *	False alarms *
1	0.6011	0.5831	0.3156	0.4494	0.1837	30.4%	0.99%
2	0.4617	0.6183	0.4281	0.5232	0.2223	43.3%	0.43%
3	0.4353	0.6428	0.4406	0.5417	0.2445	32.6%	0.12%
4	0.4223	0.6718	0.4824	0.5771	0.2498	58.5%	0.99%
5	0.4163	0.6868	0.5021	0.5945	0.2665	71.1%	0.93%
6	0.4022	0.7031	0.5010	0.6021	0.2799	65.3%	0.68%
7	0.3903	0.6961	0.5369	0.6165	0.2897	71.3%	0.87%
8	0.3860	0.7160	0.5313	0.6236	0.2864	49.8%	0.25%
9	0.3776	0.7054	0.5360	0.6207	0.2928	50.4%	0.74%
10	0.3809	0.7310	0.5496	0.6403	0.3034	79.0%	0.81%
11	0.3736	0.7361	0.5571	0.6466	0.3048	76.8%	0.68%
12	0.3663	0.7508	0.5720	0.6614	0.3131	64.0%	0.56%
13	0.3610	0.7569	0.5854	0.6711	0.3235	74.3%	0.68%
14	0.3611	0.7567	0.5815	0.6691	0.3206	80.8%	0.93%
15	0.3535	0.7535	0.5899	0.6717	0.3214	81.0%	0.99%
16	0.3554	0.7707	0.5905	0.6806	0.3305	75.9%	0.56%
17	0.3472	0.7764	0.5933	0.6848	0.3358	84.9%	0.87%
18	0.3463	0.7792	0.5876	0.6834	0.3345	68.9%	0.56%
19	0.3429	0.7781	0.5965	0.6873	0.3406	69.6%	0.31%
20	0.3395	0.7866	0.6031	0.6948	0.3406	88.3%	0.99%
21	0.3343	0.7857	0.6080	0.6969	0.3380	83.9%	0.68%
22	0.3337	0.7846	0.6104	0.6975	0.3448	85.7%	0.99%
23	0.3273	0.8000	0.6186	0.7093	0.3528	85.1%	0.50%
24	0.3234	0.7972	0.6193	0.7082	0.3503	83.5%	0.50%
25	0.3281	0.7920	0.6276	0.7098	0.3548	87.7%	0.74%
26	0.3192	0.8022	0.6245	0.7134	0.3596	89.2%	0.81%
27	0.3180	0.8031	0.6301	0.7166	0.3598	86.0%	0.62%
28	0.3180	0.8064	0.6269	0.7167	0.3613	90.1%	0.68%
29	0.3152	0.8137	0.6301	0.7219	0.3688	85.1%	0.37%
30	0.3147	0.8087	0.6360	0.7224	0.3650	89.2%	0.62%
31	0.3161	0.8091	0.6365	0.7228	0.3647	90.2%	0.81%
32	0.3129	0.8185	0.6362	0.7273	0.3672	91.7%	0.81%
33	0.3107	0.8138	0.6367	0.7253	0.3703	87.3%	0.81%
34	0.3130	0.8185	0.6377	0.7281	0.3708	91.6%	0.93%
35	0.3134	0.8141	0.6427	0.7284	0.3725	91.1%	0.87%
36	0.3105	0.8176	0.6402	0.7289	0.3721	91.3%	0.93%
37	0.3110	0.8177	0.6399	0.7288	0.3719	91.8%	0.99%
38	0.3096	0.8212	0.6412	0.7312	0.3724	86.9%	0.37%
39	0.3107	0.8205	0.6385	0.7295	0.3736	91.8%	0.93%
40	0.3106	0.8216	0.6378	0.7297	0.3738	91.5%	0.93%

* Detected = share of fire/smoke validation images detected at the threshold that keeps false alarms on fire-free images at or below 1%; False alarms = the rate at that threshold. Best epoch highlighted.

A.2 MobileNetV3-L + Faster R-CNN

MobileNetV3-L + Faster R-CNN - 40 epochs

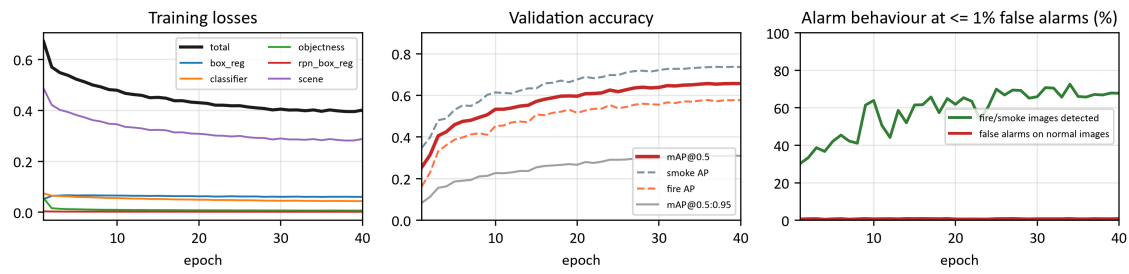


Epoch	Train loss	Smoke AP	Fire AP	mAP@0.5	mAP@.5:.95	Detected *	False alarms *
1	0.6143	0.4383	0.3027	0.3705	0.1287	44.6%	0.93%
2	0.5145	0.5207	0.3974	0.4591	0.1663	57.6%	0.99%
3	0.4908	0.5740	0.4276	0.5008	0.1997	48.0%	0.68%
4	0.4681	0.5883	0.4551	0.5217	0.2080	57.8%	0.81%
5	0.4544	0.6071	0.4667	0.5369	0.2275	53.7%	0.93%
6	0.4370	0.6107	0.4729	0.5418	0.2313	63.5%	0.87%
7	0.4265	0.6513	0.5019	0.5766	0.2437	68.5%	0.93%
8	0.4216	0.6651	0.4890	0.5770	0.2450	57.8%	0.81%
9	0.4069	0.6685	0.5366	0.6026	0.2652	66.4%	0.81%
10	0.4078	0.6874	0.5219	0.6046	0.2655	65.3%	0.74%
11	0.3899	0.6982	0.5199	0.6091	0.2706	67.5%	0.81%
12	0.3894	0.6829	0.5393	0.6111	0.2637	69.7%	0.99%
13	0.3835	0.7078	0.5441	0.6260	0.2821	73.9%	0.99%
14	0.3795	0.7161	0.5543	0.6352	0.2813	53.8%	0.50%
15	0.3743	0.7189	0.5620	0.6404	0.2917	78.1%	0.93%
16	0.3692	0.7228	0.5535	0.6381	0.2899	76.5%	0.93%
17	0.3641	0.7247	0.5707	0.6477	0.2963	75.7%	0.93%
18	0.3580	0.7244	0.5766	0.6505	0.2962	81.0%	0.87%
19	0.3567	0.7347	0.5727	0.6537	0.2981	74.2%	0.81%
20	0.3535	0.7378	0.5726	0.6552	0.3015	73.7%	0.87%
21	0.3483	0.7413	0.5770	0.6592	0.3044	80.4%	0.87%
22	0.3440	0.7493	0.5746	0.6619	0.3086	84.6%	0.81%
23	0.3423	0.7518	0.5888	0.6703	0.3140	70.8%	0.56%
24	0.3375	0.7579	0.5900	0.6740	0.3107	81.3%	0.93%
25	0.3342	0.7617	0.5973	0.6795	0.3175	82.6%	0.87%
26	0.3320	0.7568	0.6006	0.6787	0.3197	84.0%	0.62%
27	0.3317	0.7703	0.5931	0.6817	0.3218	86.5%	0.74%
28	0.3295	0.7717	0.6082	0.6899	0.3281	81.8%	0.68%
29	0.3297	0.7680	0.6100	0.6890	0.3308	85.7%	0.87%
30	0.3246	0.7759	0.6119	0.6939	0.3346	84.2%	0.87%
31	0.3227	0.7744	0.6108	0.6926	0.3345	86.1%	0.81%
32	0.3198	0.7775	0.6113	0.6944	0.3367	84.7%	0.87%
33	0.3211	0.7843	0.6161	0.7002	0.3364	81.8%	0.56%
34	0.3188	0.7801	0.6160	0.6980	0.3363	82.3%	0.74%
35	0.3204	0.7837	0.6185	0.7011	0.3361	84.1%	0.62%
36	0.3186	0.7870	0.6231	0.7050	0.3397	87.7%	0.87%
37	0.3187	0.7864	0.6199	0.7031	0.3393	84.7%	0.62%
38	0.3156	0.7865	0.6195	0.7030	0.3392	87.0%	0.99%
39	0.3167	0.7898	0.6213	0.7056	0.3397	86.9%	0.87%
40	0.3139	0.7854	0.6227	0.7040	0.3402	87.4%	0.93%

* Detected = share of fire/smoke validation images detected at the threshold that keeps false alarms on fire-free images at or below 1%; False alarms = the rate at that threshold. Best epoch highlighted.

A.3 ResNet-18 + Faster R-CNN

ResNet-18 + Faster R-CNN - 40 epochs

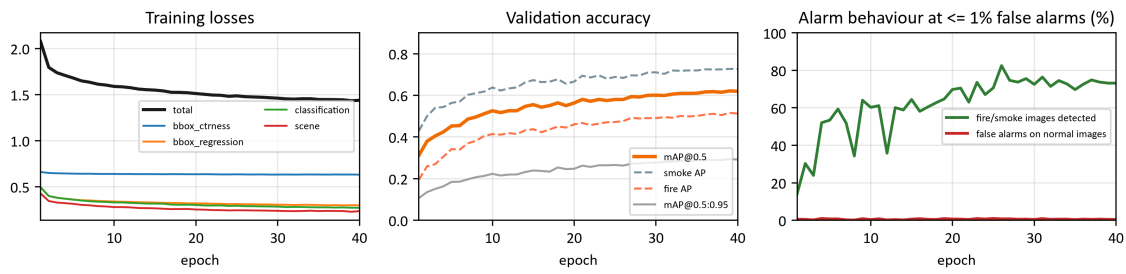


Epoch	Train loss	Smoke AP	Fire AP	mAP@0.5	mAP@.5:.95	Detected *	False alarms *
1	0.6711	0.3481	0.1625	0.2553	0.0839	30.2%	0.74%
2	0.5702	0.3977	0.2290	0.3133	0.1124	33.4%	0.87%
3	0.5482	0.4804	0.3308	0.4056	0.1557	38.7%	0.93%
4	0.5376	0.4890	0.3612	0.4251	0.1630	36.7%	0.56%
5	0.5226	0.5306	0.3874	0.4590	0.1861	42.1%	0.81%
6	0.5123	0.5518	0.3982	0.4750	0.1902	45.5%	0.93%
7	0.5000	0.5490	0.4119	0.4804	0.1938	42.3%	0.62%
8	0.4925	0.5698	0.4181	0.4940	0.2107	41.1%	0.81%
9	0.4818	0.6025	0.4106	0.5066	0.2133	61.5%	0.99%
10	0.4791	0.6138	0.4527	0.5333	0.2267	63.9%	0.81%
11	0.4669	0.6117	0.4551	0.5334	0.2260	50.7%	0.87%
12	0.4625	0.6092	0.4699	0.5396	0.2301	44.2%	0.93%
13	0.4592	0.6228	0.4742	0.5485	0.2369	58.6%	0.81%
14	0.4499	0.6335	0.4712	0.5523	0.2363	52.0%	0.99%
15	0.4513	0.6335	0.5084	0.5710	0.2540	61.5%	0.93%
16	0.4478	0.6575	0.5044	0.5810	0.2609	61.7%	0.99%
17	0.4380	0.6607	0.5160	0.5883	0.2632	65.8%	0.93%
18	0.4381	0.6709	0.5213	0.5961	0.2660	57.4%	0.87%
19	0.4326	0.6633	0.5313	0.5973	0.2698	65.0%	0.99%
20	0.4303	0.6760	0.5158	0.5959	0.2666	61.9%	0.74%
21	0.4276	0.6886	0.5272	0.6079	0.2798	65.4%	0.68%
22	0.4207	0.6824	0.5354	0.6089	0.2754	63.5%	0.74%
23	0.4205	0.6878	0.5359	0.6119	0.2813	53.8%	0.68%
24	0.4181	0.6973	0.5535	0.6254	0.2908	60.5%	0.68%
25	0.4185	0.6972	0.5380	0.6176	0.2905	70.0%	0.93%
26	0.4134	0.7097	0.5442	0.6270	0.2919	66.9%	0.93%
27	0.4101	0.7164	0.5549	0.6356	0.2963	69.5%	0.99%
28	0.4074	0.7183	0.5598	0.6391	0.2969	69.3%	0.87%
29	0.4019	0.7142	0.5571	0.6357	0.2944	65.2%	0.74%
30	0.4051	0.7211	0.5553	0.6382	0.3001	65.9%	0.87%
31	0.4015	0.7268	0.5662	0.6465	0.3032	70.8%	0.87%
32	0.4019	0.7273	0.5630	0.6452	0.3048	70.5%	0.87%
33	0.3987	0.7270	0.5712	0.6491	0.3068	65.7%	0.87%
34	0.4019	0.7312	0.5696	0.6504	0.3075	72.6%	0.99%
35	0.3965	0.7330	0.5746	0.6538	0.3088	66.0%	0.81%
36	0.4020	0.7356	0.5776	0.6566	0.3103	65.8%	0.81%
37	0.3981	0.7368	0.5709	0.6538	0.3086	67.1%	0.93%
38	0.3957	0.7356	0.5763	0.6559	0.3086	66.9%	0.87%
39	0.3955	0.7368	0.5766	0.6567	0.3097	67.9%	0.87%
40	0.4012	0.7362	0.5769	0.6566	0.3097	67.8%	0.93%

* Detected = share of fire/smoke validation images detected at the threshold that keeps false alarms on fire-free images at or below 1%; False alarms = the rate at that threshold. Best epoch highlighted.

A.4 DINOv3 ViT-Ti + FCOS

DINOv3 ViT-Ti + FCOS - 40 epochs



Epoch	Train loss	Smoke AP	Fire AP	mAP@0.5	mAP@.5:.95	Detected *	False alarms *
1	2.0771	0.4288	0.1946	0.3117	0.1067	14.1%	0.56%
2	1.7952	0.4999	0.2589	0.3794	0.1353	30.3%	0.50%
3	1.7373	0.5399	0.2695	0.4047	0.1508	24.0%	0.19%
4	1.7076	0.5436	0.3052	0.4244	0.1631	52.0%	0.99%
5	1.6797	0.5637	0.3418	0.4528	0.1843	53.5%	0.74%
6	1.6490	0.5702	0.3395	0.4549	0.1853	59.3%	0.74%
7	1.6343	0.6024	0.3697	0.4861	0.1968	51.9%	0.25%
8	1.6132	0.6108	0.3818	0.4963	0.2072	34.3%	0.12%
9	1.6032	0.6182	0.4053	0.5118	0.2131	64.1%	0.87%
10	1.5884	0.6379	0.4139	0.5259	0.2230	60.3%	0.37%
11	1.5843	0.6232	0.4112	0.5172	0.2150	61.1%	0.81%
12	1.5747	0.6331	0.4187	0.5259	0.2195	35.8%	0.19%
13	1.5598	0.6382	0.4143	0.5263	0.2202	60.1%	0.37%
14	1.5529	0.6622	0.4317	0.5469	0.2318	58.9%	0.19%
15	1.5485	0.6732	0.4374	0.5553	0.2393	64.5%	0.62%
16	1.5399	0.6633	0.4231	0.5432	0.2369	58.1%	0.87%
17	1.5247	0.6658	0.4364	0.5511	0.2337	60.5%	0.50%
18	1.5239	0.6776	0.4503	0.5639	0.2537	62.7%	0.43%
19	1.5200	0.6559	0.4454	0.5506	0.2458	64.7%	0.93%
20	1.5112	0.6661	0.4609	0.5635	0.2475	69.8%	0.68%
21	1.5036	0.6941	0.4671	0.5806	0.2624	70.5%	0.62%
22	1.4952	0.6857	0.4573	0.5715	0.2564	63.1%	0.43%
23	1.4927	0.6964	0.4639	0.5802	0.2639	73.6%	0.93%
24	1.4813	0.6797	0.4708	0.5752	0.2641	67.1%	0.68%
25	1.4862	0.6885	0.4720	0.5802	0.2655	70.7%	0.99%
26	1.4774	0.6839	0.4763	0.5801	0.2628	82.5%	0.74%
27	1.4742	0.6960	0.4921	0.5940	0.2712	74.7%	0.81%
28	1.4712	0.6954	0.4889	0.5921	0.2731	73.7%	0.56%
29	1.4661	0.7095	0.4906	0.6001	0.2767	75.6%	0.62%
30	1.4611	0.7107	0.4907	0.6007	0.2766	72.5%	0.43%
31	1.4539	0.7030	0.4952	0.5991	0.2801	76.4%	0.93%
32	1.4510	0.7193	0.4927	0.6060	0.2815	71.4%	0.50%
33	1.4550	0.7178	0.5011	0.6095	0.2850	74.6%	0.62%
34	1.4553	0.7197	0.4990	0.6093	0.2861	72.8%	0.62%
35	1.4470	0.7194	0.5011	0.6103	0.2887	69.8%	0.37%
36	1.4463	0.7251	0.5075	0.6163	0.2903	72.6%	0.62%
37	1.4455	0.7243	0.5114	0.6178	0.2898	74.9%	0.50%
38	1.4432	0.7246	0.5051	0.6148	0.2896	73.7%	0.62%
39	1.4337	0.7269	0.5156	0.6212	0.2935	73.2%	0.50%
40	1.4379	0.7267	0.5124	0.6196	0.2920	73.2%	0.43%

* Detected = share of fire/smoke validation images detected at the threshold that keeps false alarms on fire-free images at or below 1%; False alarms = the rate at that threshold. Best epoch highlighted.

Appendix B. Notes on measurement

- **Epochs.** Three models were trained for 40 epochs. The FCOS model was trained for 45; this report shows its first 40 (best 0.6212 at epoch 39). Its test result comes from its best checkpoint overall (epoch 44, validation 0.6220).
- **Test-time settings.** Part-way through the project the detector's test-time settings were changed for speed (300 proposals instead of 1,000, at most 20 boxes per image instead of 50). The DINOv3 results in this report were recorded before the change and the two CNN results after it. The change was measured to cost about 0.005 mAP, so DINOv3's lead over the CNNs is about half a point smaller than tabulated. It remains ahead by about 2 points over MobileNetV3 and 6-7 over ResNet-18.
- **Head comparison.** The two trained detectors differ in both head and backbone size; see section 7.
- **CPU timings** are from the project laptop (batch 1, 640 px, 32-bit, median of 15 runs); ranges show two measurement sessions. They indicate relative speed, not deployment speed on a GPU.
- **Alarm figures** are per image. On video an alarm additionally needs 6 detections within 15 frames, which removes most single-frame mistakes.